

موتور تشخیص کپی‌های معنایی و سرقت ادبی

TF-IDF وزن‌دار شده با SimHash و Shingling + MinHash + LSH

پروژه ۳ درس داده‌کاوی

استاد شاکری

اعضای گروه

۴۰۳۱۳۰۰۸	علی تقی‌زاده
۴۰۲۱۳۰۰۸	ستایش حاجی اسفندیاری
۴۰۲۱۳۰۰۲	زهرا اسلامی‌نیا

لینک مخزن پروژه

github.com/alitkbbi/Semantic-Plagiarism-Engine

فهرست مطالب

۳	۱	نمای کلی پروژه
۳	۲	خط لوله‌ی پیش‌پردازش متن
۴	۱.۲	مدیریت حالت‌های مرزی
۴	۳	روش اول: Shingling + MinHash + LSH
۴	۱.۳	شباهت دقیق Jaccard و هزینه‌ی آن
۵	۲.۳	MinHash، پیاده‌سازی شده از پایه
۵	۳.۳	LSH مبتنی بر باند، پیاده‌سازی شده از پایه
۷	۴	روش دوم: SimHash وزن‌دهی شده با TF-IDF
۷	۱.۴	الگوریتم
۷	۲.۴	چرا وزن‌دهی TF-IDF اهمیت دارد
۷	۵	انتخاب پارامترها
۷	۱.۵	اندازه شینگل (k)
۸	۲.۵	طول امضای MinHash (num_perm)
۸	۳.۵	باندهای LSH (num_bands)
۹	۴.۵	عرض بیت SimHash
۹	۵.۵	آستانه‌های تصمیم‌گیری
۹	۶.۵	انتخاب خودکار آستانه (--sweep)
۱۰	۷.۵	بازبینی اندازه شینگل: بازه‌های طولانی و مبهم (PAN-PC-11)
۱۰	۶	مجموعه داده‌ها
۱۰	۱.۶	مجموعه داده‌های مقیاس بزرگ پیشنهادی (غیرموجود در مخزن)
۱۱	۲.۶	پیکره‌ی نمونه (data/sample_corpus/)
۱۱	۳.۶	مجموعه داده‌ی مصنوعی جفت‌های برچسب‌دار (data/raw/quora/sample_pairs.csv)
۱۲	۴.۶	پیکره‌ی واقعی PAN-PC-11
۱۳	۷	ارزیابی تجربی
۱۳	۱.۷	نتایج سطح پیکره: مقایسه‌های اجتناب شده توسط LSH
۱۴	۲.۷	مثال‌هایی از مقایسه دو سند
۱۵	۳.۷	نتایج روی مجموعه داده‌ی جفت‌ها: دقت، فراخوانی، F1 و زمان‌بندی

۴.۷	نتایج روی پیکره‌ی واقعی PAN-PC-11	۱۶
۸	تحلیل خطا	۱۷
۱.۸	ترکیب MinHash+LSH: منفی‌های کاذب در بازنویسی‌های سنگین	۱۷
۲.۸	ترکیب MinHash+LSH و SimHash: مثبت‌های کاذب در منفی‌های دشوار	۱۷
۳.۸	نمایه خطای نامتقارن SimHash	۱۸
۴.۸	یک هشدار روش شناختی درباره هم‌پوشانی کلمات پرسشی	۱۸
۵.۸	مجموعه داده‌ی واقعی PAN-PC-11: فروپاشی فراخوانی به دلیل مبهم‌سازی، و چرایی وابسته بودن راه حل به	
۱۹	مجموعه داده	۱۹
۹	نتیجه‌گیری	۲۰

۱. نمای کلی پروژه

تشخیص اسناد تقریباً تکراری و بازنویسی‌شده، یکی از زیرمسئله‌های اصلی در سامانه‌های تشخیص سرقت ادبی، حذف اخبار تکراری، خوشه‌بندی نتایج جست‌وجو، و تشخیص پرسش‌های تکراری در سامانه‌های پرسش‌وپاسخ Q&A است. دشواری اصلی این مسئله در آن است که اسناد تقریباً تکراری، معمولاً متن کاملاً یکسان ندارند. برای نمونه، فردی که مرتکب سرقت ادبی می‌شود ممکن است ترتیب جمله‌ها را تغییر دهد، واژه‌ها را با مترادف‌هایشان جایگزین کند، یا بخش‌هایی از متن را به‌طور کامل بازنویسی نماید؛ با این حال، یک موتور جست‌وجو یا سامانه‌ی تشخیص سرقت ادبی باید همچنان بتواند تشخیص دهد که دو صفحه با بیان‌های متفاوت، محتوای مشابهی را پوشش می‌دهند.

در این پروژه، دو رویکرد کلاسیک و شناخته‌شده برای حل این مسئله پیاده‌سازی و با یکدیگر مقایسه شده‌اند:

۱. روش **Shingling + MinHash + LSH**: در این روش، هر سند به‌صورت مجموعه‌ای از k -گرام‌های واژگانی هم‌پوشان، یا همان shingle، نمایش داده می‌شود. شباهت میان دو سند با استفاده از شاخص Jaccard روی این مجموعه‌ها تعریف می‌گردد، سپس این شباهت با کمک امضاها (MinHash) به‌شکل کارآمد تخمین زده می‌شود. در نهایت، با استفاده از روش Locality-Sensitive Hashing (LSH)، جفت‌سند‌های کاندیدا پالایش می‌شوند تا در یک پیکره‌ی بزرگ، نیازی به مقایسه‌ی کامل و exhaustive همه‌ی جفت‌ها نباشد.

۲. روش **TF-IDF Weighted SimHash**: در این روش، هر سند با استفاده از یک فرایند تجمیع هش مبتنی بر علامت و وزن‌دهی شده، به یک اثرانگشت 64-bit تبدیل می‌شود. سپس شباهت میان دو سند، تنها با محاسبه‌ی فاصله‌ی همینگ میان دو عدد با طول ثابت به دست می‌آید.

هر دو خط لوله فقط با استفاده از کتابخانه‌ی استاندارد Python، به‌همراه NumPy و Pandas پیاده‌سازی شده‌اند. برای الگوریتم‌های اصلی، از هیچ کتابخانه‌ی آماده‌ی MinHash، SimHash یا LSH، مانند datasketch، استفاده نشده است. خروجی پروژه به‌صورت یک ابزار خط فرمان با نام plagiarism_engine.cli ارائه شده که شامل سه زیرفرمان compare، corpus و pairs است. برای این پروژه، هیچ رابط گرافیکی یا رابط تعاملی ترمینالی ارائه نشده و چنین رابطی نیز موردنیاز نبوده است.

۲. خط لوله‌ی پیش‌پردازش متن

پیش از آن‌که هر یک از دو موتور تشخیص شباهت روی متن اعمال شود، همه‌ی اسناد از یک خط لوله‌ی مشترک پیش‌پردازش در فایل preprocessing.py عبور می‌کنند:

۱. **نرمال‌سازی نویسه‌ها**: ابتدا نرمال‌سازی یونیکد از نوع NFKC انجام می‌شود. سپس نویسه‌های مشابه در خط عربی به شکل استاندارد فارسی آن‌ها تبدیل می‌شوند؛ برای مثال، ARABIC YEH به PERSIAN YEH و ARABIC KAF به PERSIAN KEHEH نگاشت می‌شود. همچنین حرکات عربی، یا همان اعراب‌گذاری، حذف می‌شوند؛ زیرا مشخصات پروژه، علاوه بر انگلیسی، پشتیبانی صحیح از متن فارسی را نیز الزامی کرده است.

۲. **حذف نشانه‌گذاری**: هم نشانه‌های نگارشی استاندارد ASCII و هم نشانه‌های رایج فارسی/عربی حذف می‌شوند. این نشانه‌ها شامل ویرگول عربی، نقطه‌ویرگول، علامت سؤال، علامت درصد، گیومه‌ها، نویسه‌ی سه‌نقطه، و نویسه‌ی کشیده یا tatweel/kashida هستند.

۳. **یکسان‌سازی فاصله‌ها**: فاصله‌های متوالی، شامل فاصله‌های معمولی، تب‌ها و شکست‌های خط، به یک فاصله‌ی ساده تبدیل می‌شوند.

۴. حذف واژه‌های ایست: یک فهرست کوچک و صریح از واژه‌های ایست حذف می‌شود. این فهرست هم واژه‌های انگلیسی مانند *the, is, and* و *or* را پوشش می‌دهد و هم معادل‌های فارسی آن‌ها مانند «و»، «است»، «را» و «یا». در متن انگلیسی اصلی، برخی از این واژه‌های فارسی برای حروف‌چینی به صورت لاتین‌نویسی شده آمده بودند، اما فهرست واقعی واژه‌های ایست در فایل preprocessing.py با خط فارسی ذخیره شده است.

۵. توکن‌سازی: متن نرمال‌شده بر اساس فاصله‌های خالی به دنباله‌ای از توکن‌ها تبدیل می‌شود.

۶. ساخت shingle: از روی دنباله‌ی توکن‌ها، k - گرام‌های واژگانی پیوسته ساخته می‌شوند.

۱.۲. مدیریت حالت‌های مرزی

در مشخصات پروژه، سه حالت مرزی به صورت صریح مطرح شده است. هر سه حالت، هم با سندهای اختصاصی در مسیر data/sample_corpus/ و هم با آزمون‌های واحد در فایل tests/test_engine.py بررسی شده‌اند:

- اسناد خالی: (doc_09.txt) سندهای خالی به جای آنکه باعث ایجاد استثنا شوند، یک فهرست توکن خالی و یک مجموعه‌ی shingle خالی تولید می‌کنند. طبق قرارداد، دو سند خالی مشابهت کامل دارند و مقدار $J = 1.0$ برای آن‌ها در نظر گرفته می‌شود. این تصمیم با قرارداد رایج برای حالت نامعین 0/0 در شباهت Jaccard سازگار است.
- اسناد بسیار کوتاه (doc_07.txt)، شامل پنج واژه: (اگر تعداد توکن‌های یک سند از اندازه‌ی پیکربندی‌شده‌ی shingle یعنی k کمتر باشد، به جای آنکه هیچ shingle‌ای تولید نشود، کل دنباله‌ی توکن‌ها به عنوان یک shingle واحد در نظر گرفته می‌شود. به این ترتیب، سندهای کوتاه همچنان قابل مقایسه باقی می‌مانند).
- نویسه‌های غیرمعمول: (doc_08.txt) شامل ایموجی، حروف لاتین دارای اعراب، خط تیره‌ی بلند، نقل قول‌های خمیده: همه‌ی نویسه‌های کنترلی، قالب‌بندی و جانشین یونیکد حذف می‌شوند. در عین حال، نویسه‌های فاصله با دقت حفظ می‌شوند، هرچند از نظر رده‌بندی عمومی یونیکد در همان خانواده قرار می‌گیرند. این کار باعث می‌شود ورودی‌های ناسالم یا بدقالب، باعث توقف خط لوله یا تخریب مرزهای shingle نشوند.

۳. روش اول: Shingling + MinHash + LSH

۱.۳. شباهت دقیق Jaccard و هزینه‌ی آن

اگر دو مجموعه‌ی shingle را با A و B نشان دهیم، شباهت Jaccard به صورت زیر تعریف می‌شود:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

این معیار به صورت مستقیم در فایل cli.py و در تابع exact_jaccard پیاده‌سازی شده است. از این مقدار هم به عنوان سیگنال مرجع برای راستی‌آزمایی استفاده می‌شود و هم به عنوان معیاری که MinHash برای تقریب آن طراحی شده است.

چرا مقایسه‌ی دقیق جفت‌به‌جفت از مرتبه‌ی $O(n^2)$ است؟ برای یک پیکره شامل n سند، یافتن همه‌ی جفت‌هایی که شباهتشان از یک آستانه‌ی مشخص بیشتر است، در روش ساده نیازمند محاسبه‌ی $J(A, B)$ برای تمام جفت‌های نامرتب اسناد است. تعداد این جفت‌ها برابر است با:

$$\binom{n}{2} = \frac{n(n-1)}{2}.$$

هر محاسبه‌ی منفرد Jaccard با استفاده از اشتراک‌گیری مجموعه‌های هش شده، هزینه‌ای در حد $O(|A| + |B|)$ دارد؛ اما تعداد جفت‌ها همچنان با n به صورت درجه‌دو رشد می‌کند. بنابراین، دو برابر شدن اندازه‌ی پیکره تقریباً باعث چهار برابر شدن تعداد مقایسه‌های موردنیاز می‌شود. برای پیکره‌ی نمونه‌ی ۱۱ سندی این پروژه، این مقدار فقط ۵۵ مقایسه است و ناچیز محسوب می‌شود؛ اما در یک سامانه‌ی واقعی تشخیص سرقت ادبی که مثلاً $n = 100,000$ سند را پردازش می‌کند، مقایسه‌ی کامل به حدود 5×10^9 مقایسه‌ی جفتی نیاز دارد. اجرای تکرارشونده‌ی چنین حجمی از مقایسه، از نظر محاسباتی عملی نیست. دقیقاً همین مسئله‌ی مقیاس‌پذیری است که ترکیب MinHash + LSH برای دور زدن آن طراحی شده است.

۲.۳. MinHash، پیاده‌سازی شده از پایه

روش MinHash که توسط Broder در سال ۱۹۹۷ معرفی شد، از یک واقعیت احتمالاتی ساده استفاده می‌کند: اگر h یک تابع هش تصادفی یکنواخت از خانواده‌ای مناسب باشد، آنگاه:

$$\Pr \left[\min_{x \in A} h(x) = \min_{x \in B} h(x) \right] = J(A, B).$$

با انتخاب num_perm تابع هش مستقل و ثبت کمترین مقدار هش روی مجموعه‌ی shingle‌های هر سند برای هر تابع، یک امضا با طول ثابت ساخته می‌شود. سپس نسبت جایگاه‌هایی از امضای دو سند که مقدار یکسان دارند، یک برآوردگر بدون اریب برای شباهت واقعی Jaccard است. خطای استاندارد این برآوردگر با نرخ $O(1/\sqrt{\text{num_perm}})$ کاهش می‌یابد.

جزئیات پیاده‌سازی. فایل minhash.py خانواده‌ی استاندارد هش همگانی زیر را پیاده‌سازی می‌کند:

$$h_i(x) = (a_i \cdot x + b_i) \bmod p, \quad p = 2^{61} - 1$$

که در آن p یک عدد اول Mersenne است. ضرایب (a_i, b_i) برای هر نمونه از MinHasher فقط یک‌بار و با استفاده از یک مولد تصادفی دارای بذر مشخص تولید می‌شوند. از آن‌جا که تابع داخلی $\text{hash}()$ در Python به دلایل امنیتی و از طریق PYTHONHASHSEED در هر فرایند مقداردهی تصادفی می‌شود، نمی‌توان از آن برای نگاشت رشته‌های shingle به عدد صحیح استفاده کرد. در MinHash، یک shingle یکسان باید در همه‌ی اسناد و در همه‌ی اجراها هش یکسانی داشته باشد. به همین دلیل، ابتدا از SHA-1، کوتاه‌شده به ۶۴ بیت، به عنوان هش پایه‌ی پایدار در تابع stable_hash استفاده شده و سپس خانواده‌ی هش همگانی روی آن عدد اعمال شده است.

اگر مجموعه‌ی shingle یک سند خالی باشد، امضایی شامل مقدار نگهبان بیشینه برای همه‌ی خانه‌ها تولید می‌شود. بنابراین، دو سند خالی مشابه یکدیگر در نظر گرفته می‌شوند که با قرارداد شباهت Jaccard در بخش قبل سازگار است.

۳.۳. LSH مبتنی بر باند، پیاده‌سازی شده از پایه

یک امضای MinHash با طول num_perm به num_bands باند پیوسته تقسیم می‌شود. تعداد ردیف‌های هر باند برابر است با:

$$r = \text{num_perm} / \text{num_bands}.$$

هر باند با استفاده از SHA-1 روی مقادیر خام امضا هش می‌شود. همچنین شاخص باند به عنوان نمک به هش افزوده می‌شود تا اگر مقادیر ردیف‌ها در دو باند متفاوت یکسان بودند، به اشتباه با یکدیگر برخورد نکنند. دو سند فقط زمانی به عنوان کاندیدا برای مقایسه‌ی دقیق‌تر در نظر گرفته می‌شوند که دست‌کم یکی از باندهای آن‌ها در یک سطل مشترک قرار گیرد.

چرا این روش تعداد مقایسه‌ها را کاهش می‌دهد؟ در این روش، فقط سندهایی که در یک سطل مشترک برخورد کرده‌اند با معیار دقیق Jaccard مقایسه می‌شوند. سندهایی که هیچ باند مشترکی ندارند، اساساً وارد مجموعه‌ی کاندیداها نمی‌شوند. بخش ۷ کاهش واقعی به‌دست‌آمده روی پیکره‌ی نمونه را گزارش می‌کند: کاهش 94.55٪، یعنی فقط ۳ جفت از ۵۵ جفت ممکن وارد مرحله‌ی راستی‌آزمایی شدند.

منحنی S و انتخاب پارامتر. طرح باندبندی، رابطه‌ی مشهور «منحنی S» را میان شباهت واقعی Jaccard یک جفت سند، یعنی s ، و احتمال کاندیدا شدن آن جفت ایجاد می‌کند:

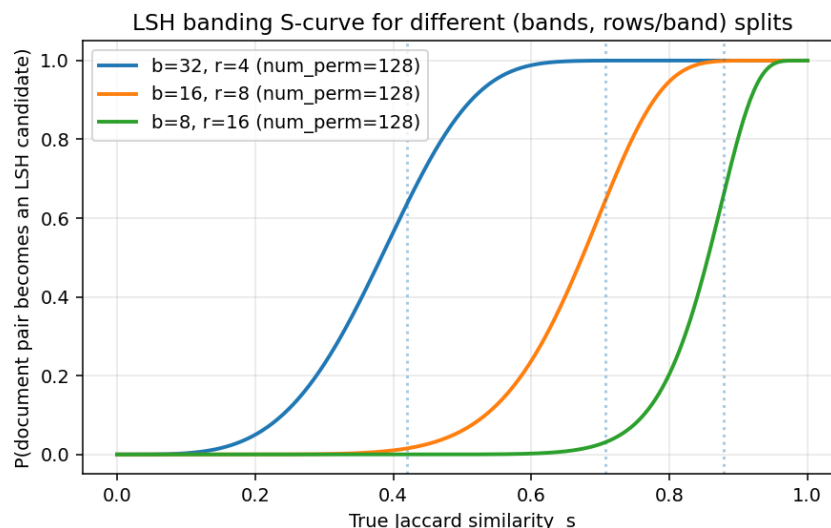
$$P(\text{candidate} | s) = 1 - (1 - s^r)^b.$$

این رابطه در فایل lsh.py و در تابع candidate_probability پیاده‌سازی شده است. مقدار این تابع برای s های پایین‌تر از یک آستانه، نزدیک به صفر و برای s های بالاتر از آن آستانه، نزدیک به یک است. تندترین ناحیه‌ی گذار آن تقریباً در نقطه‌ی زیر رخ می‌دهد:

$$s^* = \left(\frac{1}{b}\right)^{1/r}$$

که در تابع s_curve_threshold محاسبه می‌شود. شکل ۱ این منحنی را برای سه تقسیم‌بندی متفاوت (num_bands, rows_per_band) از یک امضای MinHash با 128 جایگشت نشان می‌دهد.

تعداد باندهای کمتر با اندازه‌ی بزرگ‌تر، مانند $b = 8, r = 16$ ، آستانه‌ی گذار را بالاتر می‌برد. این حالت باعث تولید کاندیداهاى مثبت کاذب کمتر می‌شود، اما منحنی آن تندتر و کم‌گذشت‌تر است و ممکن است جفت‌های با شباهت متوسط را از دست بدهد. در مقابل، تعداد باندهای بیشتر با اندازه‌ی کوچک‌تر، مانند $b = 32, r = 4$ ، آستانه‌ی گذار را پایین‌تر می‌آورد. این حالت شباهت‌های ضعیف‌تر را نیز پیدا می‌کند، اما در عوض جفت‌های کاندیدای بیشتری تولید می‌شود که باید در مرحله‌ی بعد با معیار دقیق بررسی شوند. بنابراین، انتخاب باندها یک مصالحه‌ی مستقیم و قابل تنظیم میان بازیابی در مرحله‌ی تولید کاندیدا و هزینه‌ی محاسباتی مرحله‌ی راستی‌آزمایی است.



شکل ۱: منحنی S در روش باندبندی LSH برای سه پیکربندی مختلف (bands, rows-per-band) از یک امضای MinHash با 128 جایگشت. خط‌های عمودی نقطه‌چین، آستانه‌ی تقریبی تشخیص هر پیکربندی را نشان می‌دهند: $s^* = (1/b)^{1/r}$.

۴. روش دوم: SimHash وزن‌دهی شده با TF-IDF

الگوریتم Charikar, SimHash (۲۰۰۲) رویکرد متفاوتی را اتخاذ می‌کند: به جای آنکه یک سند را به عنوان یک مجموعه در نظر بگیرد، یک اثر انگشت (fingerprint) واحد با طول ثابت را به گونه‌ای انباشت می‌کند که اسناد مشابه در فاصله همینگ (Hamming distance) کوچکی از یکدیگر قرار گیرند. این کار، جستجوی شباهت را به یک شمارش بیت ساده تبدیل می‌کند.

۱.۴. الگوریتم.

برای سندی که به صورت یک کیسه کلمات وزن‌دار (tokens) of bag (weighted) به فرم $\{(term, w)\}$ نمایش داده می‌شود:

۱. یک انباشتگر ۶۴-بیتی $V = 0$ را مقداردهی اولیه کنید.

۲. برای هر ترم (term)، یک هش ۶۴-بیتی پایدار از آن محاسبه کنید (در اینجا از BLAKE2b استفاده شده است تا از تابع درونی و تصادفی $hash()$ در پایتون اجتناب شود).

۳. برای هر یک از ۶۴ موقعیت بیتی i : اگر بیت i ام از هش ترم برابر با 1 بود، وزن w مربوط به آن ترم را به V_i اضافه کنید؛ در غیر این صورت، w را از V_i کم کنید.

۴. در نهایت، بیت i ام اثر انگشت برابر با 1 خواهد بود اگر $V_i > 0$ باشد، و در غیر این صورت برابر با 0 است.

وزن w برای هر ترم برابر با امتیاز TF-IDF آن است که از پایه در کلاس `TfidfVectorizerFromScratch` و با استفاده از فرمول استاندارد هموار شده (smoothed) زیر محاسبه می‌گردد:

$$\text{idf}(t) = \ln\left(\frac{1 + N}{1 + \text{df}(t)}\right) + 1,$$

به این ترتیب، ترمی که در تمامی اسناد پیکره آموزش دیده (fitted corpus) ظاهر می‌شود، به جای وزن دقیقاً صفر، همچنان یک وزن مثبت کوچک دریافت می‌کند و ترم‌های دیده‌نشده در زمان استنتاج، به طور منطقی به بیشینه مقدار IDF در پیکره بازمی‌گردند. شباهت بین دو اثر انگشت، هم به صورت فاصله همینگ خام (hamming_distance) و هم به صورت یک شباهت نرمال‌شده در بازه $[0, 1]$ ($= 1 - d_H/64$ hamming_similarity) گزارش می‌شود.

۲.۴. چرا وزن‌دهی TF-IDF اهمیت دارد.

در یک SimHash بدون وزن‌دهی (وزن برابر برای همه توکن‌ها)، کلمات رایج با اطلاعات کم می‌توانند بر اثر انگشت غلبه کنند و اثر آن را مخدوش سازند. وزن‌دهی مشارکت هر ترم بر اساس میزان کمیابی آن در سطح پیکره، قدرت تمایز اثر انگشت را بر روی کلماتی متمرکز می‌کند که در واقعیت موضوع یک سند را از سند دیگر متمایز می‌سازند — این رویکرد معادل پیوسته (و نه باینری) برای حذف کلمات توقف (stop-word removal) در فضای SimHash است.

۵. انتخاب پارامترها

۱.۵. اندازه شینگل (k)

مشخصات پروژه مقادیر $k \in \{3, 4, 5\}$ را برای شینگل‌بندی (shingling) توصیه می‌کند. این توصیه به طور ضمنی فرض می‌کند که اسناد در حد و اندازه یک پاراگراف هستند؛ به عنوان مثال، برای اسناد نمونه پاراگرافی در مسیر `data/sample_corpus/`

(هر کدام شامل ۵۰ تا ۷۵ توکن)، مقدار $k = 3$ مجموعه‌های شینگلی ایجاد می‌کند که به اندازه کافی بزرگ هستند (۵۰ تا ۷۰ شینگل در هر سند) تا سیگنال پایداری برای MinHash فراهم کنند، در حالی که همچنان به ویرایش‌های سطح کلمه حساس باقی می‌مانند.

متن‌های کوتاه در حد یک جمله رفتار بسیار متفاوتی دارند. جدول ۱ جاروب آستانه (threshold) sweep را برای بهترین امتیاز F1 قابل دستیابی در اندازه‌های مختلف شینگل بر روی مجموعه داده مصنوعی جفت-پرسش (بخش ۶) نشان می‌دهد؛ جایی که هر سند تنها یک پرسش ~ 10 کلمه‌ای است: با استفاده از شینگل‌های $k = 3$ ، تنها ۶ تا ۹ شینگل به ازای هر سند باقی می‌ماند، بنابراین جایگزینی تنها یک کلمه می‌تواند به طور مستقیم بخش عمده‌ای از شینگل‌های یک سند را از بین ببرد و سیگنال ژاکارد (Jaccard) را تخریب کند. کاهش k به 1 (یعنی همان اشتراک کلمات ساده بر اساس ژاکارد)، سیگنال بسیار بیشتری را برای اسناد کوتاه بازیابی می‌کند. بنابراین، ما از $k = 3$ به عنوان مقدار پیش‌فرض برای دستورات corpus/compare (اسناد پاراگرافی) و از $k = 1$ برای اجرای نمونه pairs بر روی مجموعه داده کوتاه (پرسشی) استفاده می‌کنیم، و پرچم --shingle-size را در رابط خط فرمان (CLI) قرار داده‌ایم تا بتوان آن را برای طول معمول اسناد در پیکره هدف تنظیم کرد.

جدول ۱: بهترین مقدار F1 قابل دستیابی (از طریق جاروب آستانه) بر روی مجموعه داده مصنوعی جفت-پرسش، بر اساس روش و اندازه شینگل. اندازه‌های کوچکتر شینگل، سیگنال بیشتری را برای اسناد کوتاه و جمله‌ای بازیابی می‌کنند.

Recall / F1	Precision	Best threshold	Config	Method
0.900 / 0.806	0.730	0.13	$k = 1$	MinHash+LSH
0.767 / 0.730	0.697	0.01	$k = 2$	MinHash+LSH
0.600 / 0.679	0.783	0.01	$k = 3$	MinHash+LSH
1.000 / 0.714	0.556	0.46	64-bit	TF-IDF SimHash

۲.۵. طول امضای MinHash (num_perm)

مشخصات مسئله، ۱۲۸ یا ۲۵۶ جایگشت (permutation) را توصیه می‌کند. خطای استاندارد تخمین ژاکارد در MinHash تقریباً برابر با $1/\sqrt{\text{num_perm}}$ است: حدود ۸.۸٪ برای ۱۲۸ جایگشت، و حدود ۶.۲۵٪ برای ۲۵۶ جایگشت. ما از مقدار ۱۲۸ به عنوان پیش‌فرض استفاده می‌کنیم که تعادل منطقی و مناسبی بین دقت و هزینه محاسبه امضا (هر جایگشت اضافی به معنای یک اسکن کمینه (min-scan) با مرتبه $O(|\text{shingles}|)$ به ازای هر سند است) برای پروژه‌ای با این مقیاس ارائه می‌دهد؛ با این حال، پارامتر --num-perm در دسترس است تا هر کسی که دقت بالاتری (۲۵۶ جایگشت) را با دو برابر هزینه می‌خواهد، بتواند آن را تنظیم کند.

۳.۵. باندهای LSH (num_bands)

با فرض $\text{num_perm} = 128$ ، پیش‌فرض ما $\text{num_bands} = 32$ (که نتیجه می‌دهد $r = 4$ سطر در هر باند) است. این مقادیر آستانه تشخیص تقریبی برابر با $s^* = (1/32)^{1/4} \approx 0.42$ را به دست می‌دهند (شکل ۱). این مقدار به صورت تجربی به گونه‌ای انتخاب شد که به اندازه کافی پایین باشد تا به طور پایدار و قابل اطمینان حالت “تکثیر تقریبی خفیف (light near-duplicate)” را در پیکره نمونه (فایل doc_01.txt در برابر doc_11.txt با مقدار واقعی ژاکارد ≈ 0.48) به عنوان یک کاندیدای LSH نشان دهد، و همزمان به اندازه کافی بالا باشد تا اسناد کاملاً نامرتبط اساساً هیچ‌گاه به طور تصادفی با هم تداخل (collide) نداشته باشند.

۴.۵. عرض بیت SimHash

بر اساس مشخصات، بر روی ۶۴ بیت ثابت شده است. ۶۴ بیت همچنین اندازه کلمه ماشین (word size) در تقریباً تمامی سخت‌افزارهای مدرن است که باعث می‌شود محاسبه فاصله همینگ (به شکل $\text{bin}(a \oplus b).count("1")$) کم‌هزینه و سریع باقی بماند.

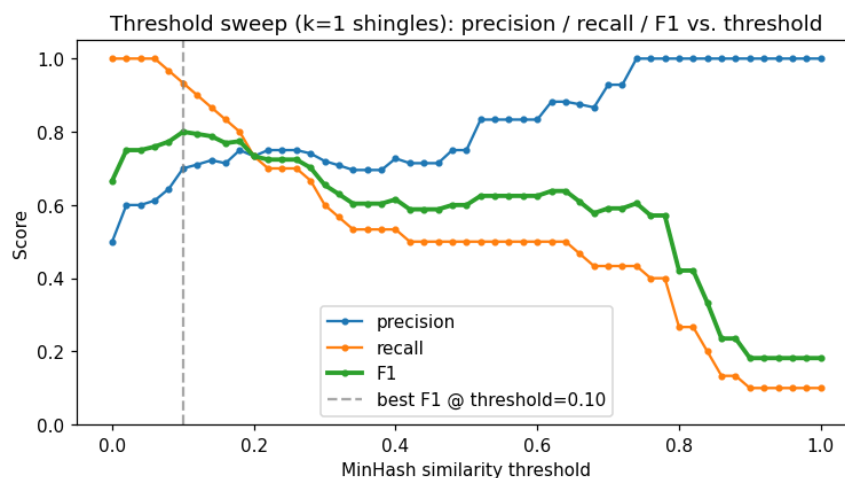
۵.۵. آستانه‌های تصمیم‌گیری

آستانه شباهت در تمامی دستورات خط فرمان به جای هاردکد شدن، به عنوان یک پرچم در دسترس قرار گرفته است (`--threshold` برای `corpus`، و `--simhash-threshold / --minhash-threshold` برای `pairs`)، زیرا همان‌طور که جدول ۱ نشان می‌دهد، نقطه عملیاتی درست به شدت وابسته به طول سند و هزینه نسبی مثبت کاذب در برابر منفی کاذب در بستر استقرار (`deployment`) `context` است (به عنوان مثال، یک ابزار تشخیص سرقت ادبی که برای ارجاع کار به بررسی انسانی استفاده می‌شود، می‌تواند نقطه عملیاتی با دقت پایین‌تر و فراخوانی بالاتری را نسبت به ابزاری که اقدام خودکار انجام می‌دهد، تحمل کند).

۶.۵. انتخاب خودکار آستانه (`--sweep`)

حدس زدن دستی یک آستانه و امید به اینکه این مقدار بر روی یک مجموعه داده جدید نیز جواب دهد، در عمل آن‌قدر غیرقابل اعتماد بود که توجیه می‌کرد به جای یک اسکریپت تحلیل یک‌بار مصرف، به عنوان یک قابلیت اصلی در ابزار پشتیبانی شود. دستور `pairs` پرچم `--sweep` را می‌پذیرد:

امتیاز شباهت هر جفت دقیقاً یک بار محاسبه می‌شود و سپس در یک شبکه (پنجره) آستانه‌ها (به طور پیش فرض: 0.00 تا 1.00 با گام‌های 0.02) از طریق تابع `sweep_thresholds` مجدداً ارزیابی می‌شود، و آستانه‌ای که F1 را بیشینه می‌کند توسط `best_threshold_row` انتخاب می‌گردد. از آنجایی که بخش عمده هزینه مربوط به امتیازدهی است و اعمال مجدد آستانه‌ها بر روی لیست امتیازات از پیش محاسبه شده از مرتبه $O(1)$ به ازای هر آستانه است، یک جاروب کامل به سختی هزینه‌ای بیشتر از یک اجرای تک‌آستانه‌ای دارد. آستانه انتخاب شده در خروجی (ستون `threshold_used`) ثبت می‌شود تا نتایج قابل تکرار باقی بمانند، و پرچم `--sweep-output` می‌تواند کل منحنی دقت/فراخوانی/F1 بر حسب آستانه را برای تحلیل‌های بعدی ذخیره کند (شکل ۲).



شکل ۲: دقت، فراخوانی و F1 به عنوان تابعی از آستانه شباهت MinHash ($k=1$)، جاروب شده بر روی مجموعه داده مصنوعی برچسب‌دار جفت‌ها. خط چین نشان‌دهنده آستانه بیشینه‌کننده F1 است که دستور `--sweep` به صورت خودکار انتخاب می‌کند.

این قابلیت به طور خاص در پاسخ به یک خطای واقعی در هنگام ارزیابی روی PAN-PC-11 (بخش ۴.۷) اضافه شد: آستانه‌های پیش‌فرض ثابتی که برای یک نوع متن تنظیم شده بودند، دقتی تقریباً بی‌نقص اما فراخوانی کاملاً فروپاشیده‌ای روی نوع دیگری از متن تولید کردند. این یک الگوی تشخیصی است که ارزش نام‌گذاری صریح را دارد، زیرا به راحتی ممکن است با معیوب بودن معیار شباهت اشتباه گرفته شود، در حالی که مشکل تنها به کالبره نبودن مرز تصمیم برمی‌گردد. در صورت وجود هرگونه آستانه قابل استفاده، جاروب کردن این مشکل را حل می‌کند؛ بخش ۴.۷ همچنین موردی را مستند می‌کند که در آن جاروب آستانه نشان می‌دهد هیچ آستانه‌ای نمی‌تواند از خط‌پایه بدیهی و پیش‌پافتاده «همیشه پیش‌بینی کن تکراری است» بهتر عمل کند، که این خود اطلاعاتی تشخیصی درباره انتخاب هایپرپارامترها محسوب می‌شود، نه خرابی ابزار.

۷.۵. بازبینی اندازه‌های شینگل: بازه‌های طولانی و مبهم (PAN-PC-11)

بخش ۱.۵ نشان داد که پرسش‌های کوتاه به اندازه شینگل کوچکتری نسبت به $k \in \{3, 4, 5\}$ که در مشخصات توصیه شده نیاز دارند، زیرا اساساً متن کافی برای تشکیل شینگل‌های زیاد وجود ندارد. مجموعه داده واقعی PAN-PC-11 (بخش ۴.۶) نیز به دلیلی دقیقاً متضاد، همین نتیجه یعنی $k = 1$ را ایجاب می‌کند: بازه‌های سرقت ادبی در آن طولانی هستند (طول بازه‌های حاشیه‌نویسی شده مشاهده شده از ۱،۴۲۱ تا ۱۸،۰۰۶ کاراکتر متغیر بود) اما مکرراً از طریق پخش کردن جایگزینی‌ها در سطح کلمه در سراسر متن، به جای بازنویسی عبارات متصل، مبهم‌سازی (*obfuscated*) شده‌اند. جایگزینی تنها یک کلمه، تمام شینگل‌های k - کلمه‌ای متداخل با آن را از بین می‌برد؛ در طول یک بازه طولانی با تعداد زیادی از این جایگزینی‌های پراکنده، این اثر انباشته می‌شود و می‌تواند اکثر اشتراکات واقعی را پاک کند، با وجود اینکه اکثریت قاطع واژگان متن همچنان مشترک هستند. شینگل‌بندی تک‌کلمه‌ای (که در عمل معادل استفاده از روش ژاکارد با کیسه - کلمات ساده است) نسبت به مکان وقوع یک ویرایش تا حد زیادی بی‌تفاوت است، هرچند که هزینه آن از دست دادن کامل آزمون ترتیب محلی کلمات خواهد بود. جدول ۶ تفاوت حاصل از این مسئله را به صورت کمی نشان می‌دهد: بر روی همان مجموعه ارزیابی ۴،۰۰۰ جفتی، برای MinHash+LSH، مقدار F_1 از ۰.۰۷۰ در $k = 3$ (آستانه‌های ثابت) به ۰.۹۲۰ در $k = 1$ (آستانه جاروب شده) افزایش می‌یابد. بنابراین، ما اندازه شینگل را به عنوان یک هایپرپارامتر وابسته به پیکره در نظر می‌گیریم که باید به ازای هر استقرار تنظیم شود، نه یک ثابت غیرقابل تغییر - مقدار $k = 3$ به عنوان پیش‌فرضی مناسب برای اسناد پاراگرافی، در دستورات corpus/compare باقی می‌ماند، اما به محض استفاده از داده‌های واقعی، هر دو ارزیابی انجام شده در این گزارش نیازمند مقدار کوچکتری بودند.

۶. مجموعه داده‌ها

۱.۶. مجموعه داده‌های مقیاس بزرگ پیشنهادی (غیرموجود در مخزن)

بر اساس مشخصات پروژه، مجموعه داده‌های خام و حجیم در این مخزن گنجانده نشده‌اند. فایل data/raw/README.md نحوه دریافت و اتصال هر یک از سه مجموعه داده‌ی پیشنهادی زیر را مستند کرده است:

- **پیکره‌ی PAN-PC-11 (پیکره‌ی سرقت ادبی PAN سال ۲۰۱۱):** مجموعه داده‌ی پیشنهادی اصلی که شامل موارد سرقت ادبی با مبهم‌سازی (Obfuscation) دستی و خودکار است و از طریق پلتفرم Zenodo در دسترس قرار دارد.
- **اسناد تکراری Stack Exchange:** جفت‌پست‌های پرسش و پاسخ که به عنوان تکراری برچسب‌گذاری شده‌اند و در هاب Hugging Face موجود هستند.
- **جفت‌محورهای پرسش Quora:** شامل بیش از ۴۰۰،۰۰۰ جفت پرسش با برچسب تکراری / غیرتکراری که در هاب Hugging Face و Kaggle قابل دسترسی است.

۲.۶. پیکره‌ی نمونه (/data/sample_corpus/)

یک پیکره‌ی ۱۱ سندی به‌صورت دستی نگارش شد تا تمامی حالات بررسی‌شده در این گزارش را به‌طور هم‌زمان بیازماید: یک پاراگراف مرجع در مورد انرژی‌های تجدیدپذیر (doc_01.txt) به همراه یک کپی واژه‌به‌واژه و عینی از آن (doc_03.txt)، شبیه‌سازی سرقت ادبی از نوع کپی-پیست، یک نسخه‌ی تقریباً تکراری با ویرایش جزئی (doc_11.txt)، جایگزینی‌های جزئی کلمات، و یک نسخه‌ی تقریباً تکراری با بازنویسی سنگین (doc_02.txt، بازنویسی کامل)؛ یک پاراگراف مرجع دوم و نامرتب در مورد اکتشاف مریخ (doc_04.txt) به همراه نسخه‌ی بازنویسی‌شده‌ی سنگین خاص خود (doc_05.txt)؛ یک پاراگراف نامرتب در حوزه‌ی آشپزی (doc_06.txt)؛ و چهار سند مربوط به حالات مرزی (اسناد بسیار کوتاه، نویسه‌های غیرمعمول، سند خالی، و متن به زبان فارسی).

۳.۶. مجموعه‌داده‌ی مصنوعی جفت‌های برچسب‌دار (data/raw/quora/sample_pairs.csv)

از آنجا که محیط بیلد مورد استفاده برای تهیه این گزارش دسترسی به شبکه نداشت، امکان دانلود مجموعه‌داده‌ی واقعی جفت‌محورهای پرسش Quora فراهم نبود. بنابراین، یک مجموعه‌داده‌ی مصنوعی ۶۰ سطر با همان ساختار و شمای استاندارد (question1, question2, is_duplicate) به‌صورت دستی ایجاد شد تا فرمان pairs بدون نیاز به هیچ پیکربندی اضافی قابل اجرا و ارزیابی باشد. این مجموعه‌داده به‌طور عمدی چهار سطح از دشواری را پوشش می‌دهد (هر کدام ۱۵ جفت، مگر مواردی که ذکر شده است):

- **تکراری‌های آسان (Light duplicates - ۱۵ جفت، برچسب ۱):** همان پرسش با یک تغییر نگارشی جزئی (برای مثال: «چگونه می‌توانم مهارت‌های صحبت کردن به زبان انگلیسی خود را بهبود بخرم؟» در برابر «چگونه می‌توانم توانایی صحبت کردن به زبان انگلیسی خود را بهبود بخشم؟»). این موارد هم‌پوشانی واژگانی بالایی دارند و حالت ساده‌ی مسئله محسوب می‌شوند.
- **تکراری‌های دشوار (Heavy duplicates - ۱۵ جفت، برچسب ۱):** همان پرسش با بازنویسی کامل و استفاده از واژگان متفاوت (برای مثال: «تفاوت بین یادگیری ماشین و یادگیری عمیق چیست؟» در برابر «یادگیری عمیق چگونه با یادگیری ماشین تفاوت دارد؟»). این موارد هم‌پوشانی واژگانی پایینی دارند و برای روش‌های مبتنی بر واژگان، چالشی جدی به شمار می‌روند.
- **منفی‌های آسان (Easy negatives - ۲۴ جفت، برچسب ۰):** موضوعات کاملاً نامرتب که هم‌پوشانی مورد انتظار برای آن‌ها نزدیک به صفر است.
- **منفی‌های دشوار (Hard negatives - ۶ جفت، برچسب ۰):** استفاده از یک قالب جمله‌ی یکسان که در آن تنها یک کلمه‌ی کلیدی محتوایی تغییر کرده است (برای مثال: «چگونه در بزرگسالی نواختن پیانو را یاد بگیرم؟» در برابر «چگونه در بزرگسالی بازی شطرنج را یاد بگیرم؟»). این موارد علی‌رغم داشتن مفهوم کاملاً متفاوت، هم‌پوشانی واژگانی بسیار بالایی دارند؛ این طراحی به‌طور هدفمند برای سنجش نقاط ضعف و حالات شکست معیارهای شباهت صرفاً واژگانی انجام شده است.

این طراحی عمدتاً به گونه‌ای نیست که یک مجموعه‌داده‌ی ساده با دقت ۱۰۰٪ ایجاد کند؛ بلکه هدف آن نمایان ساختن مصالحه‌ها و معاوضه‌های واقعی است که در بخش ارزیابی خطا مورد بحث قرار می‌گیرند؛ دقیقاً مشابه روح حاکم بر پیکره‌ی واقعی Quora که آن نیز شامل جفت‌پرسش‌های متعددی است که از نظر واژگانی بسیار شبیه هستند اما از نظر معنایی تکراری محسوب نمی‌شوند.

۴.۶. پیکره‌ی واقعی PAN-PC-11

فراتر از مجموعه‌داده‌ی مصنوعی جفت‌پرسش در بخش قبل، مجموعه‌داده‌ی پیشنهادی اصلی — یعنی پیکره‌ی واقعی PAN-PC-11 (در پلتفرم Zenodo با شناسه DOI 10.5281/zenodo.3250095) — به‌طور مستقیم داندود و مورد ارزیابی قرار گرفت. ساختار پوشه‌ها در این پیکره پس از بررسی مستقیم به‌صورت زیر تأیید شد:

```
pan-plagiarism-corpus-2011/
external-detection-corpus/
source-document/part1/ ... part23/ source-documentKKKKKKK.{txt,xml}
suspicious-document/part1/ ... part23/ suspicious-documentKKKKKKK.{txt,xml}
intrinsic-detection-corpus/
suspicious-document/part1/ ... part10/ (no source documents; not used here)
```

برای هر سند مشکوک (suspicious-document)، یک فایل XML هم‌نام وجود دارد که موارد تزریق‌شده‌ی سرقت ادبی را به‌صورت یک عنصر ویژگی (feature element) حاشیه‌نویسی کرده است:

```
<feature name="plagiarism" this_offset="..." this_length="..."
source_reference="..." source_offset="..." source_length="..." />
```

این ساختار مستقیماً و با استفاده از ابزار بازرسی اختصاصی `scripts/prepare_pan_pc11_pairs.py --inspect` روی خود پیکره تأیید شد (نه صرفاً با اتکا به شمای منتشرشده در مستندات). این ابزار پیش از ساخت هرگونه جفت‌سند، تمامی مقادیر منحصربه‌فرد موجود در عناصر `<feature name="...">` را پیمایش و استخراج می‌کند.

ساخت فایل CSV جفت‌های برچسب‌دار از روی حاشیه‌نویسی‌های خام. همان اسکریپت، قالب حاشیه‌نویسی را به ساختار استاندارد `text_a`, `text_b`, `label` تبدیل می‌کند که فرمان `pairs` از پیش انتظار آن را دارد، بدون اینکه نیازی به هیچ‌گونه تغییری در کدهای هسته‌ی موتور یعنی `src/plagiarism_engine/` باشد. این ابزار در مسیر `external-detection-corpus`، تعداد 22,186 جفت‌فایل متناظر (`.txt` و `.xml`) را شناسایی کرد (11,093 سند مشکوک و 11,093 سند منبع).

از میان این موارد، 2,000 جفت مثبت از طریق استخراج دقیق بازه‌های متناظر در سند مشکوک و سند منبع برای هر ویژگی واقعی plagiarism ساخته شد (با صفر مورد پرسش و صفر خطای تجزیه XML)، و 2,000 جفت منفی نیز با نمونه‌برداری تصادفی بخش‌هایی از اسناد نامرتب ایجاد گردید؛ در این فرایند، ۸۶۴ جفت‌سندی که در جای دیگری از پیکره رابطه‌ی سرقت ادبی برای آن‌ها حاشیه‌نویسی شده بود، به‌طور صریح کنار گذاشته شدند تا یک جفت منفی هرگز به‌طور تصادفی از دو سندی که مشخصاً با هم مرتبط هستند، استخراج نشود. در مجموع 4,000 جفت داده ساخته شد.

یک محدودیت شناخته‌شده در این روش ساخت. بازه‌های مربوط به جفت‌های مثبت از طول دقیق حاشیه‌نویسی شده در پیکره (`this_length`) استفاده می‌کنند، که تنوع بسیار بالایی دارد (دامنه‌ی مشاهده‌شده از ۱,۴۲۱ تا ۱۸,۰۰۶ کاراکتر بود)؛ اما جفت‌های منفی، برای سادگی در پیاده‌سازی، از بخش‌هایی با سقف تقریبی 1,000 کاراکتر ساخته شده‌اند. این موضوع یک عدم توازن سیستماتیک در طول متون کلاس مثبت و منفی ایجاد می‌کند که این گزارش ادعایی بر کنترل آن ندارد؛ علاوه بر سیگنال واقعی ناشی از هم‌پوشانی واژگانی، طول خود متن نیز در اینجا تا حدی پیش‌بینی‌کننده‌ی برچسب است. با این حال، یافته‌ی کیفی در بخش ۷.۵ (مبنی بر اینکه $k = 1$ تحت مبهم‌سازی، به‌طور چشمگیری از $k = 3$ بهتر عمل می‌کند) کاملاً پابرجا است، زیرا هر دو پیکربندی روی مجموعه‌داده‌ی کاملاً یکسانی ارزیابی شده‌اند — اما مقادیر مطلق دقت، فراخوانی و F1 در جدول ۶ باید با در نظر گرفتن این ملاحظه خوانده شوند، نه به‌عنوان یک اندازه‌گیری کاملاً کنترل‌شده از نظر طول. برای مشاهده راهکار پیشنهادی جهت رفع این موضوع، به بخش محدودیت‌ها مراجعه کنید.

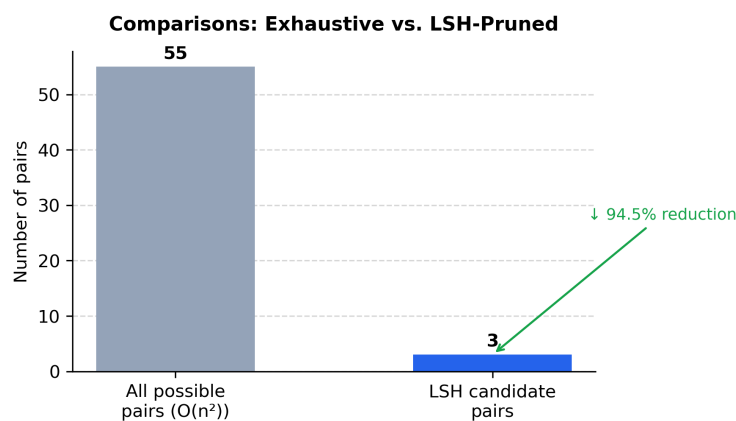
۷. ارزیابی تجربی

۱.۷. نتایج سطح پیکره: مقایسه‌های اجتناب‌شده توسط LSH

اجرای فرمان `plagiarism_engine.cli corpus` روی پیکره‌ی نمونه‌ی ۱۱ سندی (با تنظیمات `shingle_size = 3`، `num_bands = 32` و `num_perm = 128`، `threshold = 0.25`) خلاصه‌ی تحلیلی زیر را تولید می‌کند:

جدول ۲: خلاصه‌ی تولید کاندیدا توسط LSH روی پیکره‌ی نمونه‌ی ۱۱ سندی.

۱۱	تعداد اسناد بررسی شده
۵۵	کل جفت‌های ممکن، $\binom{n}{2}$
۳	جفت‌های کاندیدای تولیدشده توسط LSH
۹۴/۵۵٪	مقایسه‌های اجتناب‌شده توسط LSH
۳	جفت‌های بالاتر از آستانه (۰/۲۵) پس از راستی‌آزمایی دقیق



شکل ۳: مقایسه‌ی کامل جفت‌به‌جفت (Exhaustive) در برابر جفت‌های کاندیدای LSH که در عمل راستی‌آزمایی شدند روی پیکره‌ی نمونه.

هر سه جفت کاندیدای نمایان‌شده، در واقع موارد تکراری واقعی بودند، و راستی‌آزمایی دقیق، امتیازهای شباهت آن‌ها را به‌طور کامل تأیید کرد (جدول ۳) — این داده‌ها عیناً از فایل `outputs/candidates.csv` استخراج و بازتولید شده‌اند.

جدول ۳: Content of `outputs/candidates.csv` (with `threshold = 0.25`).

Estimate MinHash	Jaccard Exact	B Doc	A Doc
1.0000	1.0000	.txt03doc_	.txt01doc_
0.5312	0.4835	.txt11doc_	.txt01doc_
0.5312	0.4835	.txt11doc_	.txt03doc_

به‌طور خاص توجه داشته باشید که جفت‌اسناد با بازنویسی سنگین (یعنی `doc_02.txt/doc_01.txt` و `doc_05.txt/doc_04.txt`) در این جدول حضور ندارند — دلیل این پدیده در بخش ارزیابی خطاها تحلیل شده است.

۲.۷. مثال‌هایی از مقایسه دو سند

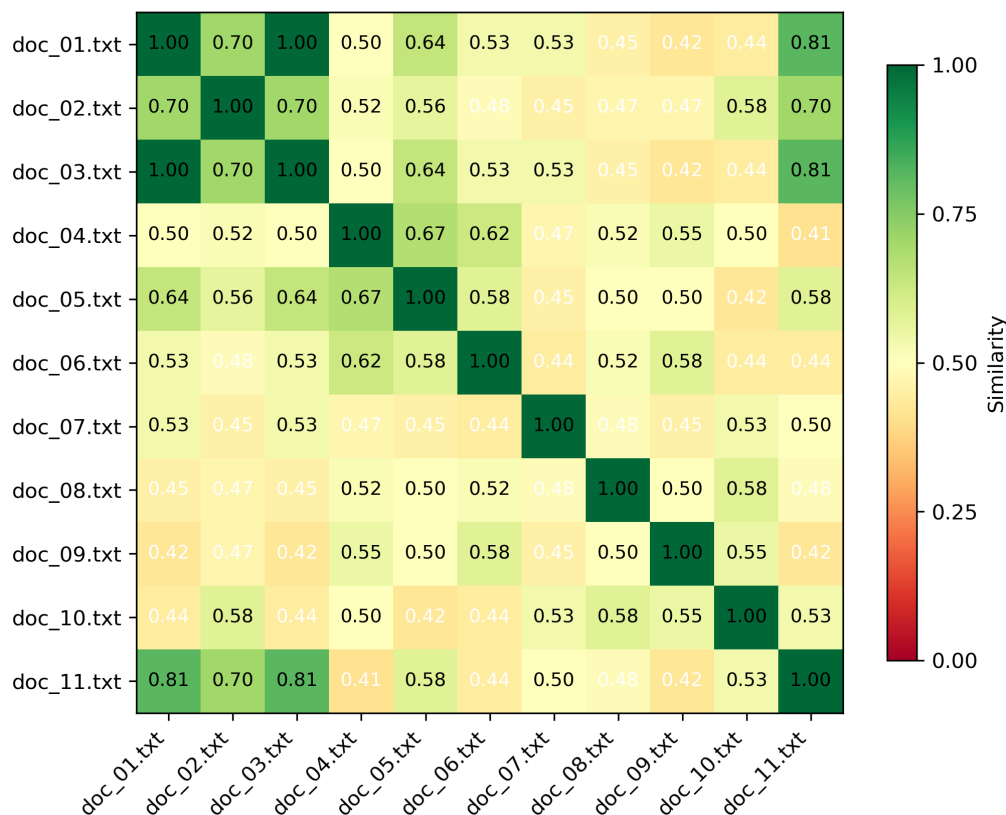
اجرای مستقیم فرمان `plagiarism_engine.cli compare` روی جفت‌های منفرد، رفتار هر دو موتور را در دو سر طیف دشواری به تصویر می‌کشد (جدول ۴).

جدول ۴: Output of the command `compare` for four sample document pairs.

Similarity SimHash	Candidate LSH	Estimate MinHash	Jaccard Exact	Pair Document
1.0000	Yes	1.0000	1.0000	copy) (verbatim 03doc_ / 01doc_
0.8125	Yes	0.5312	0.4835	edit) (minor 11doc_ / 01doc_
0.6875	No	0.0312	0.0238	rewrite) (heavy 02doc_ / 01doc_
0.6406	No	0.0156	0.0183	rewrite) (heavy 05doc_ / 04doc_

در تمامی حالات، تخمین MinHash بسیار نزدیک به شباهت دقیق جاکارد عمل می‌کند (میانگین خطای مطلق برابر با 0.0021 در تمامی ۵۵ جفت پیکره که در فایل `notebooks/exploration.ipynb` محاسبه شده است)، که این امر تأیید می‌کند برآوردگر دقیقاً مطابق با پیش‌بینی تئوری رفتار می‌کند. در مقابل، شباهت همینگ در الگوریتم SimHash تحت شرایط بازنویسی سنگین، رفتار بسیار ملایم‌تر و منعطف‌تری دارد (افت به بازه‌ی ۰/۶۴ تا ۰/۶۹ به جای فروپاشی به نزدیک صفر)؛ چرا که این روش روی یک کیسه‌ی کلمات وزن‌دار کار می‌کند و نه روی k -گرام‌های پیوسته و دقیق؛ بنابراین تا زمانی که واژگان زیربنایی تا حد زیادی حفظ شده باشند، این الگوریتم در برابر تغییر ترتیب جملات و تغییرات نگارشی محلی مقاوم باقی می‌ماند (Graceful Degradation).

SimHash Pairwise Similarity (Hamming)



شکل ۴: نقشه‌ی حرارتی (Heatmap) شباهت همینگ در روش TF-IDF Weighted SimHash برای تمام جفت‌های پیکره‌ی نمونه‌ی ۱۱ سندی (محاسبه‌ی IDF روی کل پیکره انجام شده است).

۳.۷. نتایج روی مجموعه داده‌ی جفت‌ها: دقت، فراخوانی، F1 و زمان‌بندی

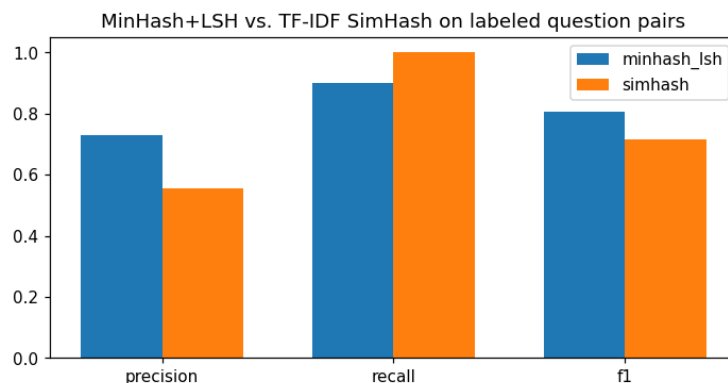
جدول ۵ نتایج مندرج در فایل outputs/metrics.csv را بازتولید می‌کند که با اجرای فرمان زیر:

```
python -m plagiarism_engine.cli pairs \
--pairs data/raw/quora/sample_pairs.csv \
--text-col-a question1 --text-col-b question2 --label-col is_duplicate \
--shingle-size 1 --minhash-threshold 0.13 --simhash-threshold 0.46 \
--output outputs/metrics.csv
```

و با استفاده از پارامترهای تنظیم‌شده در بخش ۵ (اندازه شینگل $k = 1$ برای متن‌های با طول پرسش، و آستانه‌های انتخاب‌شده از جاروب آستانه در جدول ۱) به دست آمده است.

جدول ۵: re-Exact dataset. synthetic -pair60 the on counts result and score. F1 Recall. Precision. file. outputs/metrics.csv the from production

TP/FP/FN/TN	Score F1	Recall	Precision	Method
27/10/3/20	0.8060	0.9000	0.7297	MinHash+LSH
30/24/0/6	0.7143	1.0000	0.5556	SimHash TF-IDF



شکل ۵: مقایسه دقت، فراخوانی و امتیاز F1 برای هر دو موتور روی مجموعه داده‌ی مصنوعی جفت‌های برجسب‌دار.

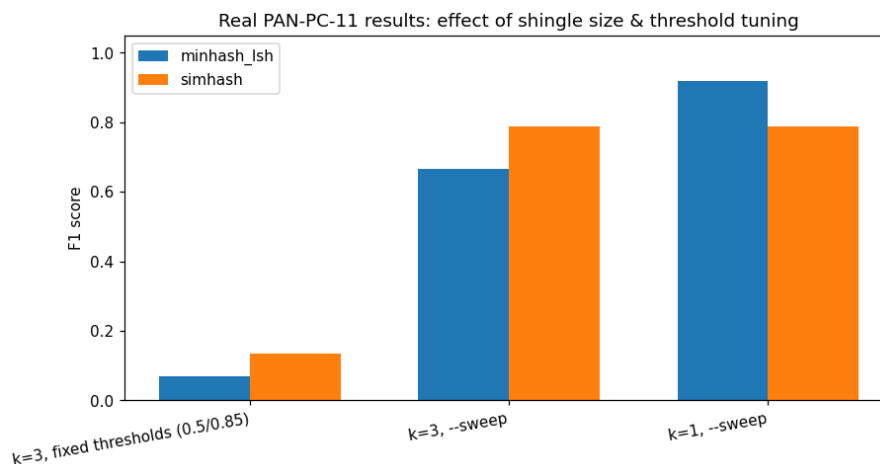
در آستانه‌های کالیبره‌شده‌ی مربوط به هر روش، خط لوله‌ی MinHash+LSH به دقت و امتیاز F1 بالاتری دست می‌یابد، در حالی که SimHash فراخوانی کامل (۱۰۰٪) را به قیمت تولید موارد مثبت کاذب (False Positives) بسیار بیشتر به دست می‌آورد — این الگوریتم ۲۴ مورد از ۳۰ منفی واقعی را به اشتباه به عنوان تکراری علامت‌گذاری می‌کند. همچنین SimHash در این آزمایش تقریباً ۵/۶ برابر سریع‌تر به ازای هر جفت عمل می‌کند؛ زیرا مقایسه‌ی فاصله‌ی همینگ میان دو عدد ۶۴-بیتی از مرتبه‌ی زمانی $O(1)$ است، در حالی که مقایسه‌ی MinHash از مرتبه‌ی $O(\text{num_perm})$ (در اینجا ۱۲۸) مقایسه به ازای هر جفت) به علاوه‌ی هزینه‌ی ساخت هر امضا از روی مجموعه‌ی شینگل‌های آن است. این زمان‌بندی‌های مطلق صرفاً جنبه‌ی نمایشی دارند و روی یک مجموعه داده‌ی ۶۰ سطر در یک سیستم اشتراکی اندازه‌گیری شده‌اند؛ برتری کیفی مورد انتظار (کم‌هزینه‌تر بودن مقایسه‌ی اثرانگشت نسبت به مقایسه‌ی امضا به ازای هر جفت) به طور کلی برقرار است، اما هیچ‌یک از این اعداد نباید به عنوان یک بنچمارک در مقیاس صنعتی تلقی شوند. مزیت ساختاری بسیار بزرگ‌تر خط لوله‌ی MinHash+LSH — یعنی اجتناب کامل از تعداد مقایسه‌های $O(n^2)$ در حجم کاری جست‌وجوی پیکره از طریق هرس کاندیداها توسط LSH — در اجرای فرمان pairs اصلاً مورد سنجش قرار نمی‌گیرد، زیرا این فرمان جفت‌های از پیش تشکیل‌شده را ارزیابی می‌کند، نه اینکه کل یک پیکره را جست‌وجو کند؛ آن مزیت بزرگ به طور مستقل در بخش ۱.۷ اثبات شد.

۴.۷. نتایج روی پیکره‌ی واقعی PAN-PC-11

مجموعه‌داده‌ی ۴,۰۰۰ جفتی PAN-PC-11 که در بخش ۴.۶ ساخته شد، به سه روش مورد ارزیابی قرار گرفت تا اثر اندازه شینگل از اثر تنظیم و کالیبره‌سازی آستانه تفکیک شود:

جدول ۶: evaluation real-world -pair4,000 the on SimHash and MinHash+LSH of Performance by (suggested threshold and size shingle fixed settings: two comparing (PAN-PC-۱۱), dataset shingle. and threshold swept versus specifications) project

Score F1	Recall	Precision	Threshold Applied	Method	Configuration
0.0695	0.0360	1.0000	0.50	MinHash+LSH	(0.5/0.85) fixed, $k = 3$
0.1352	0.0725	1.0000	0.85	SimHash TF-IDF	(0.5/0.85) fixed, $k = 3$
0.6667	1.0000	0.5000	0.00	MinHash+LSH	--sweep with, $k = 3$
0.7889	0.7445	0.8389	0.58	SimHash TF-IDF	--sweep with, $k = 3$
0.9200	0.8745	0.9706	0.10	MinHash+LSH	--sweep with, $k = 1$
0.7889	0.7445	0.8389	0.58	SimHash TF-IDF	--sweep with, $k = 1$



شکل ۶: امتیاز F1 روی مجموعه‌ی ارزیابی واقعی PAN-PC-11 بر اساس اندازه شینگل و استراتژی انتخاب آستانه. ردیف مربوط به SimHash در هر دو ستون $k = 1$ و $k = 3$ کاملاً یکسان است، چرا که پرچم --shingle-size تنها خط لوله‌ی MinHash+LSH را پارامتردهی می‌کند؛ عرض n -گرام توکن‌ها در الگوریتم SimHash یک تنظیمات مستقل و (در حال حاضر) غیرقابل دسترسی از طریق خط فرمان است که به‌طور پیش‌فرض روی تک‌گرام (Unigram) قرار دارد (بخش محدودیت‌ها).

سه یافته‌ی تحلیلی و کلیدی از این نتایج استنتاج می‌شود: اول: آستانه‌های ثابتی که از پیکره‌ی دمو (با طول پاراگراف) به این پیکره منتقل شدند، به‌شدت برای این مجموعه‌داده کالیبره‌نشده هستند: هر دو روش به دقت کامل (۱۰۰٪) می‌رسند اما فراخوانی آن‌ها زیر ۷/۵٪ است؛ این وضعیت، نشانه‌ی بارز و کلاسیک انتخاب یک آستانه‌ی بیش‌ازحد بالا است، نه نشانه‌ی خراب بودن معیار شباهت (به بخش انتخاب خودکار آستانه مراجعه کنید). دوم: صرف جارب کردن آستانه (در حالی که همچنان $k = 3$ است) برای MinHash+LSH کافی نیست — آستانه‌ی بهینه‌ی بیشینه‌کننده‌ی F1 که پیدا می‌شود برابر با 0.00 است؛ این یعنی فرایند جارب کشف می‌کند که هیچ آستانه‌ای در $k = 3$ برای روش MinHash+LSH نمی‌تواند از خط پایه‌ی بدیهی «همه را تکراری پیش‌بینی کن» (که با توجه به نرخ ۵۰ درصدی کلاس مثبت، امتیاز $F_1 = 0.667$ دارد) بهتر عمل کند. این امر نشان می‌دهد که امتیازهای شباهت در $k = 3$ اساساً هیچ سیگنال قابل استفاده‌ای در این مجموعه‌داده حمل نمی‌کنند. در مقابل، الگوریتم SimHash حتی در حالت $k = 3$ نیز یک نقطه‌ی عملیاتی منطقی و کارآمد را بازیابی می‌کند ($F_1 = 0.789$);

چرا که بازنمایی و اثرانگشت آن از همان ابتدا مستقل از ترتیب (Order-independent) است. سوم: کاهش اندازه شینگل به $k = 1$ ، عملکرد بسیار قدرتمندی را برای خط لوله‌ی MinHash+LSH بازیابی می‌کند ($F_1 = 0.920$)، که بهترین نتیجه در میان تمام پیکربندی‌های این گزارش است)، که این موضوع مکانیزم تحلیلی شرح داده شده در بخش ۷.۵ را تأیید می‌کند: مبهم‌سازی‌های پراکنده در پیکره‌ی PAN-PC-11 آسیب به مراتب بیشتری به k -گرام‌های پیوسته وارد می‌کنند تا به هم‌پوشانی ساده‌ی کلمات. نکته‌ی بسیار حائز اهمیت این است که رتبه‌بندی نسبی این دو روش میان این مجموعه داده و مجموعه داده‌ی مصنوعی جفت‌پرسش در جدول ۵ کاملاً معکوس می‌شود (در آنجا فراخوانی SimHash برتری داشت؛ اما در اینجا امتیاز F_1 در روش MinHash+LSH مسلط است) — بخش ارزیابی خطاها به‌طور مفصل توضیح می‌دهد که چرا طول سند و سبک مبهم‌سازی عامل اصلی این پدیده هستند، نه اینکه یکی از روش‌ها به‌طور مطلق و ذاتی «بتر» از دیگری باشد.

۸. تحلیل خطا

این بخش به بررسی نمونه‌های مشخصی از دسته‌بندی‌های اشتباه می‌پردازد که مستقیماً از آزمایش‌های فوق استخراج شده‌اند، و نشان می‌دهد که این موارد درباره‌ی حالت‌های شکست (Failure Modes) هر روش چه حقایقی را آشکار می‌سازند.

۱.۸ ترکیب MinHash+LSH: منفی‌های کاذب در بازنویسی‌های سنگین

هر سه مورد منفی کاذب MinHash+LSH در آستانه‌ی تنظیم‌شده ($k = 1$ ، آستانه 0.13)، جفت‌های heavy_duplicate (تکراری‌های سنگین) هستند — یعنی جفت‌های تکراری واقعی که تقریباً با واژگانی کاملاً متفاوت بیان شده‌اند:

- «فواید مدیریت‌شده منظم چیست؟» در برابر «تمرین منظم مدیریت‌شده چگونه به یک فرد کمک می‌کند؟» (شباهت 0.102، زیر آستانه)
- «چگونه سگ مضطرب خود را در طول طوفان رعد و برق آرام کنم؟» در برابر «وقتی سگم از رعد و برق می‌ترسد برای تسکین او چه کار کنم؟» (شباهت 0.078)
- «تفاوت بین ویروس و باکتری چیست؟» در برابر «ویروس‌ها چه تفاوتی با باکتری‌ها دارند؟» (شباهت 0.094)

این همان نقطه ضعف ساختاری و مورد انتظار شباهت ژاکارد (Jaccard) مبتنی بر شینگل (shingle) است: این روش معیاری از هم‌پوشانی واژگانی (Lexical) است، نه هم‌ارزی معنایی. هنگامی که در یک بازنویسی، اکثر کلمات محتوایی با مترادف‌ها جایگزین می‌شوند (تسکین دادن ↔ آرام کردن، ترس از رعد و برق ↔ در طول طوفان)، مجموعه‌های شینگل می‌توانند تقریباً مجزا از هم شوند، حتی با وجود اینکه یک خواننده‌ی انسانی بلافاصله این سوالات را به عنوان موارد تکراری تشخیص می‌دهد. همین پدیده توضیح می‌دهد که چرا جفت اسناد با بازنویسی سنگین در پیکره‌ی نمونه (doc_02.txt/doc_01.txt) و (doc_05.txt/doc_04.txt) علیرغم اینکه بازنویسی‌های واقعی یکدیگر بودند، به عنوان کاندیداهای LSH در جدول ۳ ظاهر نشدند.

۲.۸ ترکیب MinHash+LSH و SimHash: مثبت‌های کاذب در منفی‌های دشوار

هر ۶ جفت منفی دشوار (hard-negative) (الگوی یکسان، تنها با تغییر یک کلمه محتوایی) توسط هر دو روش به اشتباه به عنوان تکراری طبقه‌بندی شدند، برای مثال:

- «بهترین راه برای پختن برنج روی اجاق گاز چیست؟» در برابر «بهترین راه برای پختن کینوا روی اجاق گاز چیست؟» — شباهت MinHash+LSH برابر 0.695؛ شباهت SimHash برابر 0.688.

- «چگونه در بزرگسالی نواختن پیانو را یاد بگیرم؟» در برابر «چگونه در بزرگسالی شطرنج بازی کردن را یاد بگیرم؟» - شباهت MinHash+LSH برابر 0.734؛ شباهت SimHash برابر 0.672.

این جفت‌ها در ۸ کلمه از ۹ کلمه (یا بیشتر) کاملاً یکسان هستند و فقط در یک کلمه‌ی محتوایی که کل معنی سوال را تغییر می‌دهد متفاوتند. هیچ روش کاملاً واژگانی - چه مبتنی بر هم‌پوشانی مجموعه (MinHash) و چه مبتنی بر کیسه کلمات وزن‌دار (SimHash) - نمی‌تواند «برنج» را از «کینوا» به عنوان موضوعات متفاوت تشخیص دهد؛ بلکه هر دو صرفاً ابعاد متمایزی در فضای توکن هستند و بدون توجه به اینکه از نظر معنایی چقدر متفاوتند، به یک اندازه در میزان اختلاف نقش دارند. تشخیص این دسته از خطاها نیازمند بازنمایی‌ای است که معنای کلمه را درک کند (مانند تعبیه‌های کلمه یا جمله - embeddings)، که صراحتاً خارج از محدوده این پروژه است.

۳.۸. نمایه خطای نامتقارن SimHash

موارد مثبت کاذب SimHash (۲۴ مورد از ۳۰ منفی واقعی، شامل بسیاری از جفت‌های easy_negative با موضوعات کاملاً بی‌ربط، مثلاً «تفاوت بین یادگیری ماشین و یادگیری عمیق چیست؟» در برابر «تفاوت بین ویروس و باکتری چیست؟»، شباهت 0.734) پرشمارتر هستند و به منفی‌های دشوار محدود نمی‌شوند. ما این مسئله را به یک انتخاب طراحی خاص در خط لوله‌ی ارزیابی pairs نسبت می‌دهیم: برای مستقل نگه‌داشتن ارزیابی هر جفت، فایل evaluation.py آمارهای IDF را فقط روی دو سند موجود در همان جفت (بینید run_simhash_pipeline) برازش (Fit) می‌کند، نه روی کل مجموعه‌داده. با وجود تنها دو سند، فرکانس سند (Document Frequency) یک عبارت فقط می‌تواند مقادیر ۱ یا ۲ را بگیرد - تقریباً هیچ مفهوم مفیدی از کمیابی در سطح کل پیکره برای بهره‌برداری وجود ندارد، بنابراین وزن‌های به‌دست‌آمده تقریباً یکنواخت می‌شوند و عملکرد SimHash به سمت یک مقایسه‌ی کیسه کلمات بدون وزن تنزل می‌یابد. در نتیجه، هر دو سوال انگلیسی با طول مشابه و ساختار جمله‌ی مشابه (مانند «چیست...»، «چگونه...») صرفاً به دلیل کلمات تابعی (Function Words) و توکن‌های ساختاری مشترک، به یک شباهت پایه‌ی همینگ (Hamming) در حد متوسط دست می‌یابند، که آستانه 0.46 - که برای به حداکثر رساندن F_1 در کل مجموعه‌داده تنظیم شده است - آن‌قدر سخت‌گیرانه نیست که بتواند آن‌ها را در هر حالتی فیلتر کند. این یک هزینه‌ی مستقیم و قابل‌اندازه‌گیری از طراحی ارزیابی جفت‌به‌جفت است و در شکل ۴ در نقطه مقابل قابل مشاهده است؛ جایی که SimHash یک بار روی کل پیکره‌ی نمونه ۱۱ - سندی برازش شده است: در آنجا، جفت اسناد با موضوعات نامرتب (مانند پاراگراف آشپزی در برابر پاراگراف اکتشاف مریخ) به طور مشهودی شباهت پایین‌تری نسبت به جفت‌های مرتبط نشان می‌دهند، زیرا IDF در سطح پیکره اسناد کافی برای کاهش وزن کلمات رایج عمومی در اختیار دارد. توصیه: برای استقرار در مقیاس پیکره، چنانچه یک مجموعه‌داده‌ی برچسب‌دار از جفت‌ها از یک پیکره‌ی زیربنایی منسجم استخراج شده است، IDF مربوط به SimHash را یک بار روی کل پیکره (همان‌طور که دستور corpus می‌تواند انجام دهد و در دفترچه یادداشت اکتشافی انجام شده است) برازش کنید، نه به‌صورت جفت‌به‌جفت.

۴.۸. یک هشدار روش‌شناختی درباره هم‌پوشانی کلمات پرسشی

یک اثر ثانویه و کوچکتر: با شینگل‌های $k = 1$ (یونی‌گرم)، کلمات پرسشی رایج مانند «how»، «do»، «what»، و «is» در لیست حداقل کلمات توقف (Stop Words) (بخش ۲) قرار نمی‌گیرند، زیرا آن لیست به جای کلمات خاص پرسشی، کلمات تابعی بسته را هدف قرار می‌دهد. بنابراین، دو سوال نامرتب که هر دو با «چگونه من...» شروع می‌شوند، به دلیل کلمات الگوی جمله، مقداری هم‌پوشانی پایه‌ی یونی‌گرمی دارند که در چندین مثبت کاذب منفی - آسان MinHash نقش دارد (مثلاً «چگونه یک لپ‌تاپ کند را تعمیر کنم؟» در برابر «گیاهان چگونه غذای خود را می‌سازند؟»، شباهت 0.172، درست بالای آستانه 0.13). یک لیست کلمات توقف دامن - خاص برای پیکره‌های پرسش‌وپاسخ (با گسترش لیست پایه با کلمات پرسشی) احتمالاً این دسته از خطاها را کاهش می‌دهد؛ ما در اینجا این کار را انجام ندادیم تا لیست کلمات توقف را به عنوان یک پیاده‌سازی وفادار و حداقلی از نیازمندی‌های صریح مشخصات پروژه (کلمات «and»، «is»، «the»، «or» و معادل‌های فارسی) نگه داریم و آن را روی یک مجموعه‌داده‌ی خاص بیش‌برازش (Overfit) نکنیم.

۵.۸. مجموعه داده‌ی واقعی PAN-PC-11: فروپاشی فراخوانی به دلیل مبهم‌سازی، و چرایی وابسته بودن راه‌حل به مجموعه داده

بخش ۴.۷ نشان داد که با استفاده از آستانه‌های ثابت، فراخوانی MinHash+LSH در $k = 3$ به 3.6% سقوط کرد. به بیان دقیق‌تر: با میانگین طول قطعه در حدود چند هزار کاراکتر، یک شینگل ۳- کلمه‌ای تنها زمانی زنده می‌ماند که هر سه کلمه‌ی متوالی بدون ویرایش باقی بمانند. استراتژی مبهم‌سازی (Obfuscation) «مصنوعی» PAN-PC-11 (طبق مستندات خود پیکره) جایگزینی‌هایی را اعمال می‌کند که به جای تمرکز در یک نقطه، در سراسر متن توزیع شده‌اند. بنابراین، حتی یک نرخ متوسط جایگزینی کلمه می‌تواند باعث شود که تقریباً هیچ ۳- گرمی (3-gram) به صورت کامل دست‌نخورده باقی نماند – مقدار ژاکارد دقیق برای ۳- شینگل می‌تواند به شکل موجهی برای جفتی که انسان بلافاصله آن را همان متن (اما ویرایش شده) تشخیص می‌دهد، به زیر 0.05 برسد. این دقیقاً همان حالت شکست اساسی است که برای بازنویسی‌های سنگین در بخش ۱.۸ مستند شده است، با این تفاوت که بسیار شدیدتر است زیرا مبهم‌سازی PAN-PC-11 متراکم‌تر بوده و قطعات طولانی‌تر هستند (فرصت‌های بیشتری برای افتادن یک کلمه جایگزین شده در داخل هر شینگل وجود دارد). شینگل‌بندی یونی‌گرمی از این مشکل عبور می‌کند زیرا یک کلمه‌ی جایگزین شده تنها یک عنصر را از مجموعه‌ای متشکل از صدها یا هزاران عضو حذف می‌کند، نه اینکه زنجیره‌ای از ۳- گرم‌های هم‌پوشان را نابود کند.

چرا راه‌حل (استفاده از k کوچکتر) به جای اینکه جهان‌شمول باشد، به مجموعه داده وابسته است: کاهش k حساسیت به ترتیب کلمات و ساختار عبارت را از بین می‌برد. برای پیکره‌ای که در آن اسناد تقریباً تکراری غالباً کپی‌های کلمه به کلمه یا با ویرایش کم هستند (مانند پیکره‌ی نمونه‌ی پاراگراف‌محور در بخش ۱.۷)، $k = 3$ جفت‌های نامربوط را به درستی رد می‌کند دقیقاً به این دلیل که نیازمند تطابق پیوسته است؛ تغییر آن به $k = 1$ در آنجا باعث افزایش نرخ مثبت‌های کاذب در میان اسناد با موضوع مشابه – اما – متمایز می‌شود (همان سازوکاری که به عنوان حالت شکست «منفی دشوار» در بخش ۲.۸ برای جفت سوالات مستند شد). بنابراین، اندازه مناسب شینگل تابعی از نحوه تفاوت اسناد تقریباً تکراری یک پیکره نسبت به منبع آنهاست – ویرایش‌های متمرکز نیازمند k بزرگتر هستند؛ ویرایش‌های پراکنده نیازمند k کوچکتر – به همین دلیل است که این گزارش با آن به عنوان یک فرایارامتر (Hyperparameter) قابل تنظیم رفتار می‌کند (بخش ۷.۵) نه اینکه یک مقدار ثابت «صحیح» را پیشنهاد دهد.

چرا رتبه‌بندی SimHash و MinHash+LSH در میان مجموعه داده‌ها جابجا می‌شود: در جفت سوالات ترکیبی کوتاه (جدول ۵)، وزن‌دهی TF-IDF در SimHash سیگنال قابل استفاده‌ی بیشتری از متون بسیار کوتاه نسبت به ژاکارد k – گرم استخراج کرد؛ در قطعات متنی بلند PAN-PC-11 (جدول ۶)، MinHash+LSH با یونی‌گرم، SimHash را شکست می‌دهد. ما این موضوع را به معنای برتری مطلق هیچ‌کدام از روش‌ها نمی‌دانیم – هر دو روش در هسته‌ی خود معیارهای واژگانی از نوع کیسه‌ای (Bag-of-something) هستند، و اینکه کدام یک پیروز می‌شود به طول سند و الگوی فضایی ویرایش‌ها در آن پیکره‌ی خاص بستگی دارد. این دقیقاً استدلالی در حمایت از پیاده‌سازی و مقایسه دو روش مستقل است، به جای اینکه همان‌طور که در مشخصات پروژه نیز خواسته شده، تنها به یکی بسنده کنیم.

تعامل با محدودیت عدم تطابق طول (بخش ۴.۶): از آنجایی که قطعات مثبت می‌توانند بسیار طولانی‌تر از بخش‌های کوتاه شده‌ی منفی باشند، بخشی از تفکیک‌پذیری ظاهری در جدول ۶ ممکن است صرفاً ناشی از خود طول باشد تا مقاومت در برابر مبهم‌سازی. این مسئله نتیجه‌گیری نسبی مبنی بر اینکه $k = 1$ سیگنال بیشتری را نسبت به $k = 3$ تحت مبهم‌سازی پراکنده بازایی می‌کند، تغییر نمی‌دهد (هر دو پیکربندی در این مجموعه داده‌ی یکسان و با عدم تطابق طول برابر ارزیابی شده‌اند)، اما به این معنی است که مقدار مطلق $F_1 = 0.920$ نباید به عنوان یک اندازه‌گیری تمیز و کنترل‌شده – از – نظر – طول برای عملکرد کشف سرقت ادبی در دنیای واقعی ذکر شود.

۹. نتیجه‌گیری

ما از پایه، هم یک خط لوله LSH + MinHash + Shingling و هم یک خط لوله SimHash وزن‌دار با TF-IDF را برای کشف تکراری‌های معنایی و سرقت‌های ادبی پیاده‌سازی کردیم، هر دو را از طریق یک CLI با سه دستور (corpus, compare, pairs) در دسترس قرار دادیم و آن‌ها را بر روی سه مجموعه داده ارزیابی کردیم: یک پیکره دست‌چین شده از اسناد، یک مجموعه داده ترکیبی برچسب‌دار از جفت-سوالات و پیکره واقعی سرقت ادبی PAN-PC-11 شامل 22,186 جفت. هرس کاندیداهای LSH از انجام 94.55% مقایسه‌های دوبه‌دو در پیکره نمونه جلوگیری کرد در حالی که هیچ یک از موارد شبه‌تکراری واقعی بالای آستانه را از دست نداد، و تخمین ژاکارد MinHash به خوبی ژاکارد دقیق را دنبال کرد (میانگین خطای مطلق 0.0021). در ارزیابی جفت-سوالات ترکیبی، MinHash+LSH در یک نقطه‌ی عملیاتی تنظیم‌شده به دقت (Precision) و F1 بالاتری (0.730 / 0.806) رسید، در حالی که SimHash به فراخوانی (Recall) کامل (1.000) اما با دقت پایین‌تر (0.556) دست یافت. در پیکره واقعی PAN-PC-11، شینگل $k = 3$ که در مشخصات پروژه توصیه شده بود در ترکیب با آستانه‌های تنظیم‌نشده باعث فروپاشی فراخوانی MinHash+LSH به 3.6% علیرغم دقت کامل شد - که این یک شکست در کالبراسیون آستانه بود، نه یک شکست الگوریتمی - این مسئله انگیزه اضافه کردن قابلیت انتخاب خودکار آستانه (--sweep) به ابزار را ایجاد کرد؛ با کاهش اندازه شینگل به $k = 1$ برای مقابله با مبهم‌سازی پراکنده در سطح کلمات PAN-PC-11، معیار MinHash+LSH به $F_1 = 0.920$ ارتقا یافت، که قوی‌ترین نتیجه در این گزارش است (با در نظر گرفتن هشدار مستند شده درباره عدم تطابق طول). رتبه‌بندی نسبی این دو روش بین ارزیابی سوالات - کوتاه و قطعات - ادبی - طولانی معکوس شد، که نشان می‌دهد هیچ‌یک به‌طور مطلق برتر نیستند: طول سند و الگوی فضایی ویرایش‌ها تعیین می‌کنند که کدام بازنمایی سیگنال بیشتری را حفظ می‌کند. تحلیل خطا، حالت‌های شکست مشخص و قابل تکراری را برای هر دو روش شناسایی کرد - بازنویسی‌های جایگزین‌کننده‌ی واژگان و مبهم‌سازی پراکنده (منفی‌های کاذب)، و جایگزینی‌های تک-کلمه‌ای با حفظ الگو (مثبت‌های کاذب) - که روش‌های معنایی (مبتنی بر تعبیه) را به عنوان گام بعدی و طبیعی که فراتر از محدوده این پروژه است، توجیه می‌کند.